

Homework for L03

name: FINICHIU Eduard - Adelin

group: 1241 EB

FILS CTI-(E)

REFLECTION:

For this homework, I chose my own dataset and experimented with different prompts to see how the model responded. At first, it was interesting to notice how much the wording of a prompt could change the output.

Even small changes, like adding a tone or asking for a specific format, created very different results. This helped me understand how important prompt design is when working with language models.

Choosing my own dataset also made the task feel more personalized and relevant. I learned that the model performs best when prompts are clear and detailed. Overall, the assignment helped me develop better prompt-crafting skills and gave me more confidence working with AI models.

What I did:

- I uploaded a PDF with car specs for 3 models of luxury cars. The PDF contains mostly engine and performance related info. **At first, I used different prompts for different queries. After that I have made a comparison between prompts, using the same query.**

At first, I used the following prompt:

```
[13]
✓ Os
def generate_answer (query, context_chunks):
    context = "\n\n".join([c['text'] for c in context_chunks ])

    prompt = f""" You are a car salesman who delivers information fast and with great interest in selling cars.
    Context:
    {context}
    Question: {query}
    Answer: """

    inputs = tokenizer (prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        **inputs,
        max_new_tokens=50,
        pad_token_id=tokenizer.eos_token_id
    )
    answer = tokenizer.decode(outputs[0][len(inputs.input_ids[0]):], skip_special_tokens=True)
    return answer.strip()
```

Step 8. RAG pipeline

The next output was produced:

RESULTS....

```
query = "How powerful is the BMW X5?"
result = rag_query(query)
print(result["answer"])
```

... The BMW X5 xDrive30d is powered by a 3.0-liter inline-6 turbo diesel engine that produces 245 horsepower and 540 Newton-meters of torque.

The second prompt leaned more towards are “boored salesman” approach:

```
def generate_answer(query, context_chunks):
    context = "\n\n".join([c['text'] for c in context_chunks ])

    prompt = f""" You are boored and not interested in responding.
    Context:
    {context}
    Question: {query}
    Answer: """

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        **inputs,
        max_new_tokens=50,
        pad_token_id=tokenizer.eos_token_id
    )
    answer = tokenizer.decode(outputs[0][len(inputs.input_ids[0]):], skip_special_tokens=True)
    return answer.strip()
```

The answer was according to the prompt (booring response):

RESULTS....

```
query = "What is the bore x stroke for BMW E39 530D?"
result = rag_query(query)
print(result["answer"])
```

*** The bore x stroke for the BMW E39 530D is 84.0 × 88.0 mm.

This answer is correct because the provided text explicitly states under the "Engine & Performance" section of the BMW

Notice how I changed the query. *I did this because the “bore x stroke” can be very booring information.* I wanted to see if the AI tells anything nice about it. It seems that the AI decided to be bored and give us directly the info. He stated that the Info is corect, because he took it from the PDF. His attitude was something like: “this is it, it is 100% ok, now please let me go”

Take a look at this query with the same prompt:

RESULTS....

```
query = "Is BMW E39 a good car for me?"
result = rag_query(query)
print(result["answer"])
```

*** The BMW E39 5 Series is a well-regarded model, known for its refined driving dynamics, spacious interior, and strong performance. However, whether it's a good car for you depends on your specific needs and preferences.

Third prompt chosen was more extreme, to highlight prompt impact on answers:

```
prompt = f"""Speak like an overly enthusiastic used-car salesman. Use energetic language,
lots of exclamation marks, and persuasive phrases. Always sound excited.
```

On the **next page** you can see a massive difference:

RESULTS....

```
[22]
✓ 12s query = "Is BMW E39 a good car for me?"
      result = rag_query(query)
      print(result["answer"])

... The BMW E39 530d is an absolute *legend*! 🚗 It's got a massive 2.9L diesel engine with 184-193 horsepower and a *jaw-dro
```

Let's try the same query with different powerful prompts!

Prompt:

```
prompt = f"""Answer with very short sentences. No extra words. No explanations.
Just direct statements.
```

Response was spon on with the prompt:

RESULTS....

```
[24]
✓ 12s query = "Is BMW E39 a good car for me?"
      result = rag_query(query)
      print(result["answer"])

... No. BMW E39 is not a good car for you. It is outdated. It has poor fuel economy. It is not safe. It has high emissions. It
```

A new prompt :

```
prompt = f"""Respond like a chill person talking casually on the street.
Use slang and relaxed tone, but keep it understandable.
```

Response:

RESULTS....

```
query = "Is BMW E39 a good car for me?"
result = rag_query(query)
print(result["answer"])

... No, the BMW E39 is not the right car for you. You're likely thinking of the BMW X5 xDrive30d from 2012,
```

Beside these prompts, I have also tried different prompts like "Speak only with CAPSLOCK" or "Speak like a very old lady". I observed that Prompts need to be very detailed in order to obtain high performance. Prompts have a lot of impact on the final result. This made me think more about how we talk to Ai's and how (maybe) the Ai's capabilities can be limited by our prompting skills.